

Wk06: Data Engineering

How to Make Data Queryable

PPOL 5206 | Massive Data Fundamentals | Spring 2026

The Question That Defines Data Engineering

Scenario: A data set is stored on multiple CDs, organized by year, each with a separate schema.

Scenario: The universe of patents is stored in multiple files on a remote cloud server.

None of it is queryable yet.

Data engineering answers one question: How does data get from where it is to where it needs to be, in a form that's useful?

This lecture covers the concepts, formats, history, and tools that answer that question.

What Data Engineering Is (and Isn't)

Data Scientists ask:

"What does this data tell us?"

Data Engineers ask:

"How do we make this data available, reliable, and fast?"

DE is not analysis. DE is not modeling. DE is designing the infrastructure and processes that make analysis and modeling **possible**.

Policy professionals need to understand both because you'll oversee systems, evaluate vendor proposals, and make architecture decisions that constrain what analysis is possible downstream.

A Brief History of Storing Data

Every era's file format was a solution to the previous era's problem

Era 1: Fixed-Width Flat Files (1960s–1980s)

When storage cost dollars per kilobyte, every byte mattered. No delimiters, no headers, no metadata.

```
24510003245000218902431000428700019
```

The Data Dictionary (a separate document):

Positions	Field	Example
1–5	FIPS Code	24510
6–11	Population	003245
12	Race Code	0

This is schema-on-read in its purest form. The file has no opinion about its own structure. Meaning is applied at read time by whoever holds the dictionary.

The Data Dictionary as the Original Semantic Layer

The Census data dictionary separated three things that we still separate today:

Then (1970s)	Now (2020s)	Concept
Raw bytes on tape	Raw Parquet files in S3	Storage
Data dictionary	Glue Catalog / dbt / LookML	Semantic Layer
Printed report	Dashboard / API	Presentation

This three-layer separation [storage, meaning, presentation] is the organizing principle of modern data architecture. Tools change; this pattern persists.

Modern "semantic layer" tools (dbt metrics, Looker, Snowflake) formalize this: they define what "Q3 revenue" means once, independent of storage or visualization.

SIPP Data in 1984

Source: [US Census Bureau, Survey of Income and Program Participation, 1984](#)

SIPP 1984 FULL PANEL RESEARCH FILE DICTIONARY

D SUSEQNUM 6 1
Sequence number of person
Primary sort key

D ROT 1 7
Rotation group number

D SU-ID 9 8
Scrambled identifier equivalent to the
combination of PSU, segment number, and
serial number

D PP-ENTRY 2 17
Edited entry address ID
Address ID of the household that this
person belonged to at the time this
person first became part of the sample.

D PP-PNUM 3 19
Edited person number

D SU-TOTPP 2 22
Total number of person records
for this sample unit

D HH-ADDID 64 24
Address ID - This field
identifies the household this person
lived in this month.

```
LIBNAME sas '.';  
FILENAME INPUTDAT 'sipp84fp.dat';  
DATA sas.sipp84fp;  
INFILE INPUTDAT PAD LRECL=10760;  
INPUT  
SUSEQNUM $ 1 -6  
ROT       7 -7  
SU_ID $ 8 -16  
PP_ENTRY $ 17 -18  
PP_PNUM $ 19 -21  
SU_TOTPP       22 -23  
HH_ADD01 $ 24 -25  
HH_ADD02 $ 26 -27  
HH_ADD03 $ 28 -29  
HH_ADD04 $ 30 -31  
HH_ADD05 $ 32 -33  
HH_ADD06 $ 34 -35  
.
```

Era 2: Delimited Files, {CSV} (1970s–present)

The innovation: put delimiters between values and column names in the first row. The schema partially travels with the data.



What CSV solves

- Universal readability
- Human-editable in any text editor
- Every tool can ingest it
- Still the most common format in government data exchange



What CSV doesn't solve

- No data types (is "123" a number?)
- No nesting or complex structures
- Delimiter collisions ("Washington, D.C.") (TSV)
- No compression

```
fips,population,race_code,income  
24510,003245,0,48902
```


Era 3: Self-Describing Formats {XML & JSON}

Data that knows what it is. Every value carries its label.

XML (1990s)

- Born from SGML; strict and validatable
- Schema enforcement via XSD
- Still used: IRS filings, HL7/FHIR, SOAP, and every Office file
- Verbose; lots of overhead per value

JSON (2000s)

- Born from JavaScript; lighter than XML
- Supports nesting and arrays natively
- Lingua franca of web APIs (data.gov, Census, BLS)
- Keys repeated every record; expensive at scale

The tradeoff: self-description adds overhead. Every JSON record repeats key names. At scale, this verbosity costs real money (Athena charges per byte scanned).

Era 4: Columnar & Binary Formats

As datasets grew to billions of rows, human-readable formats became bottlenecks. Binary formats trade readability for performance.

Parquet

- Columnar, compressed, splittable
- Created 2013 (Twitter + Cloudera)
- Analytical workhorse of data lakes
- ~90% less data scanned vs CSV

Avro

- Row-oriented with embedded schema
- Optimized for fast writes
- Schema evolution (add fields safely)
- Good for streaming landing zones

ORC

- Optimized Row Columnar
- Created for Hive ecosystem
- Similar to Parquet in practice
- Parquet vs ORC = ecosystem choice

Same query, same data: CSV = \$5.00 per scan | Parquet = \$0.50 per scan → That's a budget conversation, not just a technical one.

Era 5: Table Formats {Delta Lake & Iceberg}

The data lake promised flexibility. The reality was "data swamp." These metadata layers add warehouse reliability to lake storage.

What Delta/Iceberg add on top of Parquet files:

- ✓ **ACID Transactions** No more partial writes or corrupted reads
- ✓ **Time Travel** Query data as it existed last Tuesday; audit trails
- ✓ **Schema Enforcement** Reject data that doesn't match expected structure
- ✓ **Schema Evolution** Add columns without breaking existing queries
- ✓ **Partition Management** Automatically optimize how data is organized on disk

The Lakehouse: Flexibility of a lake + reliability of a warehouse. This is where Databricks and Snowflake converge. We'll work with both later in this course.

Why Parquet Files Never Change

Parquet files are write-once. Once saved to S3, nobody modifies them. This sounds wasteful — it's actually a powerful design choice.

Compression

Data is final; encoder applies aggressive dictionary encoding, run-length encoding, and bit-packing. No reserved space for future edits.

Concurrency

Ten analysts querying the same file? No locks, no conflicts. The file can't change out from under them.

Caching

S3 replicates an immutable object across zones without consistency worries. Same bytes everywhere, forever.

Compare to a live database: PostgreSQL reserves free space in every page for future updates, maintains row version visibility for transactions, and periodically runs VACUUM to reclaim dead tuples. All that overhead disappears when the file is just... done.

But if files never change ... how do you update data?

How Updates Work Without Changing Files

Delta Lake and Iceberg never modify Parquet files. Instead, they maintain a transaction log; a metadata layer that tracks what's current.

1

Original Table

Three Parquet files in S3.
Log says: files 1, 2, 3 are current.

2

Update 500 Rows

Engine writes new file 4.
Log entry: "file 2 → replaced by
file 4."

3

Query Reads Log

Uses files 1, 3, 4. Old file 2 still
exists but is skipped.

4

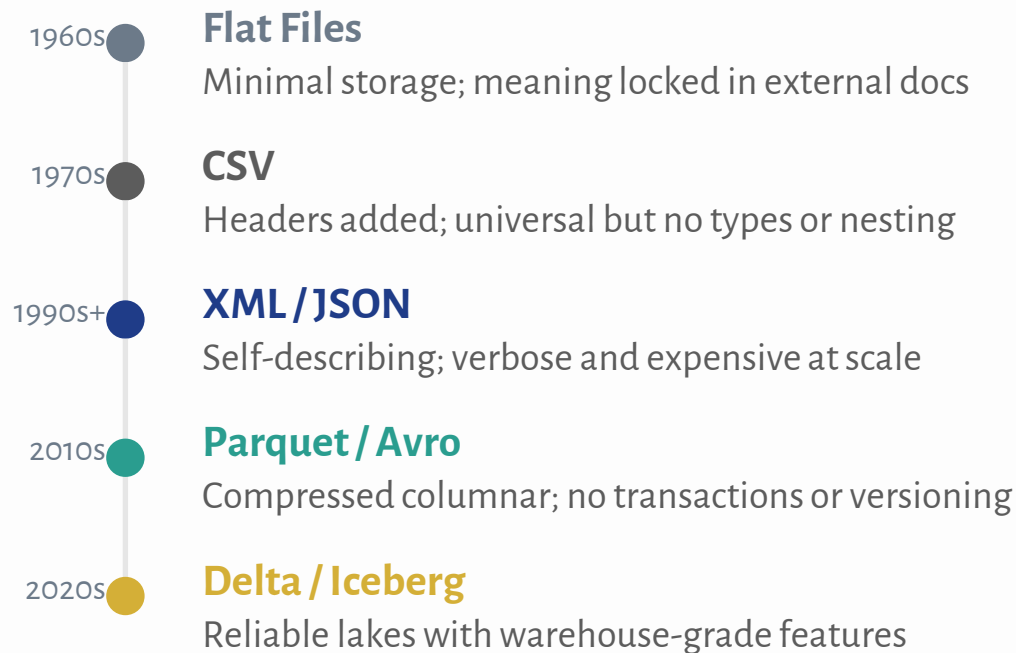
Time Travel

Query as of yesterday? Log
resolves to files 1, 2, 3. History
preserved.

Compaction: Over time, small files accumulate. Periodically, the system rewrites them as fewer large files and updates the log. Garbage collection for data lakes.

The data (Parquet files) knows nothing about versions. The meaning (what's current, what's deleted) lives in the metadata layer. Same separation of concerns as the Census data dictionary.

Each Generation Solved the Previous One's Problem



None of these replaced the previous ones entirely. Government agencies today have all of them coexisting.

The Right Data Shape for the Right Data Question

The same data, stored differently, can better answer different questions

Two Ways to Organize Data, Two Different Goals

OLTP: Row-Oriented

Optimized for:

Fast reads/writes of individual records

The question it answers:

"Show me everything about grant application APP-2024-00847."

Examples:

Benefits enrollment, case management, EHR, any operational database

OLAP: Column-Oriented

Optimized for:

Scanning large volumes of a few columns

The question it answers:

"Average grant amount by program across 10 million records?"

Examples:

Redshift, Snowflake, Athena on Parquet, any data warehouse

OLTP optimizes for fast writes. OLAP optimizes for fast reads at scale. You generally can't have both; and that tension is why data engineering exists.

Reading Less, Not Reading Faster

The win isn't disk speed — it's how much useless data the engine reads.

Row Storage — 50-chapter book per record. You need one chapter.

```
[Amy,28,NYC,75k,...46 more] [Bob,34,DC,92k,...46 more] ...
```

→ **Must read all 50 fields × 10M rows to get 1 field. That's 98% waste.**

Column Storage — All salaries in one volume. Grab it. Done.

```
names: [Amy,Bob,Carol...] salaries: [75k,92k,88k...] ...48 cols skipped
```

→ **Read only 1/50th of the data. 49 columns never leave storage.**

But: SELECT * WHERE id='APP-00847' → row storage wins. One read, all 50 fields.

Athena: \$5/TB scanned. CSV ≈ \$5.00 | Parquet ≈ \$0.10 — pay for columns you touch.

Schema-on-Write vs Schema-on-Read

Schema-on-Write (Warehouse)

- Define schema before loading
- Data must conform or is rejected
- Guarantees quality and query speed
- Rigid: adding a field means altering the table, updating ETL, reloading

Schema-on-Read (Lake)

- Store raw data as-is
- Apply structure when you query
- Flexible: any question reshapes differently
- Risk: no quality guarantees until you look

Policy implications

Schema-on-write = slower onboarding of new data sources but higher trust in results.

Schema-on-read = faster ingestion but more burden on analysts to validate.

Agencies make this choice, and it shapes what analysis is possible and how much you can trust it.

The Data Engineering Lifecycle

Data engineering is the plumbing that connects raw data sources to analytical consumers

The Data Engineering Lifecycle

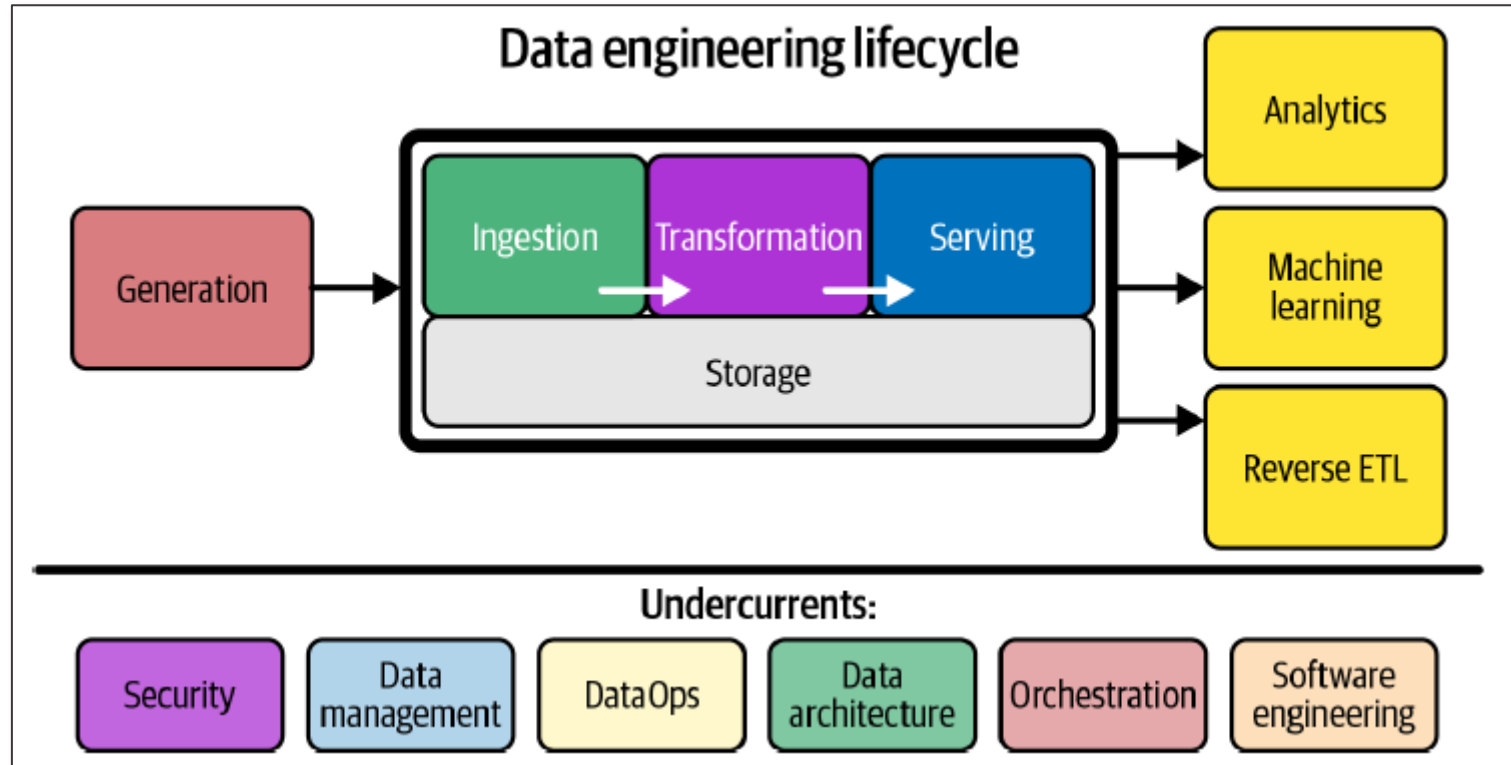


Undercurrents: Security · Data Management · DataOps · Architecture · Orchestration · Software Engineering

Ingestion: Extract from sources (APIs, files, databases) into a landing zone. Key decisions: batch vs streaming, full vs incremental, push vs pull.

Transformation: Clean, standardize, deduplicate, enrich, reshape. Fix data quality problems and reshape for analytical use cases.

Serving: Make transformed data available: SQL tables for analysts, dashboards for leadership, APIs for applications, data shares for partners.



Ingestion: Getting Data Into the System

Sources in policy contexts:

Administrative records, survey data, API endpoints (Census, BLS, data.gov), partner agencies, IoT/sensor data, documents and PDFs

Key decisions at ingestion:

Batch vs Streaming?

Most policy work is batch (nightly, weekly, quarterly)

Push vs Pull?

Does the source send to you, or do you fetch it?

Full vs Incremental?

Reload everything, or just what changed?

What format?

Determines all downstream options

Where it lands: typically a data lake (S3) as the initial landing zone. Raw data preserved in original form.

Data Lakes: Store Everything, Structure Later

A centralized repository for raw data at any scale, in any format. In AWS, this is S3.

Why policy orgs adopt lakes

Agencies receive data from dozens of sources in dozens of formats. Requiring conformity before storage means months of onboarding per source. Lakes accept everything immediately.

The "data swamp" risk

Without governance — naming conventions, folder organization, access controls, catalogs — a lake becomes a dumping ground where nobody can find or trust anything.

S3 organization best practices:

- Separate prefixes (folders) per dataset
- Consistent format within each prefix
- Partition by date when appropriate (year=2024/month=01/)
- Never mix raw and processed data in the same path

Transformation: From Raw to Ready

Raw data is almost never ready for analysis. Transformation includes:

Cleaning Handling nulls, fixing encoding, correcting data types

Standardizing Consistent date formats, unit conversions, code mappings

Deduplication Removing redundant records when merging sources

Enrichment Joining with reference data (adding state names to FIPS codes)

Reshaping Pivoting, unnesting, aggregating to the right grain

Two kinds of transformation: fixing data quality problems (every pipeline needs this) vs reshaping for a specific analytical use case (context-dependent). Both are necessary and often happen in different pipeline stages.

ETL vs ELT: Where Does Transformation Happen?

	ETL (Extract → Transform → Load)	ELT (Extract → Load → Transform)
Transform where?	Separate processing server	Inside the warehouse/lake
Raw data in warehouse?	No: only clean data enters	Yes: raw data loaded first
Ingestion speed	Slower (transform first)	Faster (load immediately)
Privacy controls	Better (strip sensitive fields before loading)	Requires warehouse-level access controls
Historical raw data?	Lost unless archived separately	Preserved as historical archive
Maintenance	More systems to maintain	Fewer systems

The judgment call: ETL gives cleaner warehouses and better privacy controls. ELT gives faster ingestion and preserves raw data. Cloud tools are narrowing the performance gap, but the architectural choice still matters for governance.

Serving: The Last Mile to Decision-Makers

Different consumers need different things:

Policy Analysts	Queryable tables (SQL, Athena, Snowflake)
Leadership	Dashboards and reports (QuickSight, Tableau, Power BI)
Applications	APIs (REST endpoints, data feeds)
Partner Agencies	Governed data shares (Snowflake sharing, S3 cross-account)
The Public	Open data portals (data.gov, agency websites)

The serving layer is where access controls, data classification, and privacy compliance become concrete. Who can see PII? Which fields are redacted for public datasets? How fresh does the data need to be?

From Concepts to Tools

AWS services as implementations of the data engineering lifecycle

AWS Services Mapped to the Lifecycle

Lifecycle Stage	AWS Service	What It Does
Storage (Lake)	S3	Scalable object storage for raw and processed data
Cataloging	Glue Data Catalog	Central metadata registry (the modern data dictionary)
Discovery	Glue Crawlers	Automatically infer schema from files (schema-on-read)
Transformation	Glue ETL Jobs	Managed Spark-based data transformation
Querying	Athena	Serverless SQL directly on S3 data
Warehousing	Redshift	Managed columnar data warehouse
Orchestration	Step Functions	Automate and schedule pipeline execution

Today's focus: S3 → Glue → Athena as a complete serverless pipeline.

The Data Catalog: A Modern Data Dictionary

Remember the Census data dictionary? A data catalog does the same thing for a data lake, but:

- ✓ **Machine-readable** Tools like Athena query it automatically
- ✓ **Auto-generated** Crawlers infer schema from data
- ✓ **Centralized** One catalog serves multiple query engines
- ✓ **Includes lineage** Where did this data come from? When was it last updated?

The Glue Data Catalog stores: database names, table names, column names and types, S3 locations, serialization format, partition keys.

Without a catalog, your data lake is just a file system. With one, it's a queryable, governed data platform.

Glue Crawlers: Automated Schema Inference

A crawler walks S3 prefixes, samples files, infers types, and registers tables in the catalog.

1. Scan target S3 path
2. Group files by location (each folder = potential table)
3. Detect file format (CSV, JSON, Parquet, etc.)
4. Sample rows to infer data types
5. Write table definitions to the catalog



This is exactly the same logic as the `mini_glue_crawler.py` demo — just running at cloud scale in a managed service. You've already seen how it works under the hood.

Schema-on-read, automated. The Glue Crawler is the Census data dictionary, written by a machine.

Athena: Serverless SQL on S3

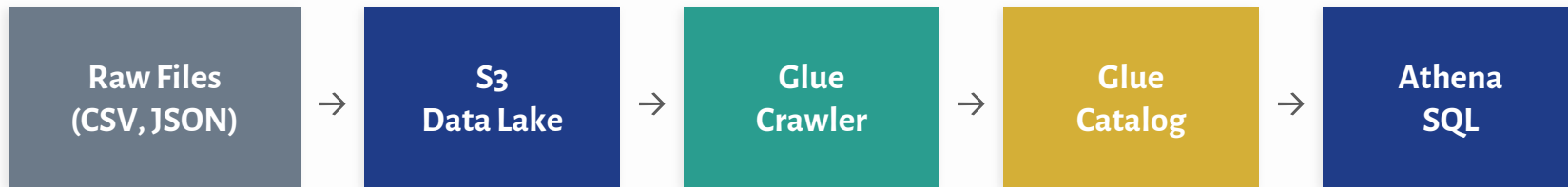
Run standard SQL directly against files in S3, using the Glue Catalog for table definitions. No servers to provision, no data to load.

Engine	Presto/Trino distributed query engine under the hood
Execution	Spin up compute → scan S3 → return results → shut down
Cost model	\$5 per TB of data scanned

File format matters: same query on Parquet vs CSV costs ~10× less because Parquet is compressed and columnar, Athena skips irrelevant columns entirely.

Policy relevance: Athena is how many agencies do ad-hoc analysis on data lake contents without maintaining a full warehouse. Low cost, no infrastructure, standard SQL.

The Complete Serverless Pipeline



Each step maps to a concept we've covered:

S3 = The data lake (ingestion/storage)

Glue Crawler = Automated schema-on-read (cataloging)

Glue Catalog = The modern data dictionary (semantic layer)

Athena = Serverless analytical querying (serving)

This is what you'll build in today's lab.

Moving Between Formats

Data Delivery to Data Analysis and Back Again

Data Arrives in One Shape; Analysis Needs Another

Government data arrives from web APIs as JSON. It needs to become structured tables. Results may need to become JSON again for APIs or partners.

Schema-on-Read

Query JSON directly; impose structure at query time. Flexible but slower.

Schema-on-Write

Convert JSON to Parquet/tables upfront. Rigid but fast queries.

Round-trip

Ingest JSON → analyze as tables
→ export as JSON for APIs.

The choice depends on: query frequency, cost sensitivity, schema stability, and what downstream consumers expect.

JSON in SQL: Flattening and Nesting

Flattening (JSON → structured):

```
SELECT
  json_extract_scalar(raw, '$.name') AS name,
  json_extract_scalar(raw, '$.address.city') AS city
FROM raw_json_table;
```

Nesting (structured → JSON): build JSON from columns for API output or data sharing with partners.



Key point:

JSON handling is conceptually portable across every modern database. The syntax varies (Athena vs Snowflake vs PostgreSQL), but the operations (extract, flatten, nest) are the same everywhere. Understanding the operation matters more than memorizing one vendor's functions.

Live Demo:

What's Inside an Office File?

Every .docx, .xlsx, and .pptx is a ZIP archive of XML files.

XML in the Wild: Office Files Deconstructed

When we unzip a .xlsx, we find:

xl/worksheets/sheet1.xml

The actual cell data (content)

xl/styles.xml

All formatting, stored separately (presentation)

docProps/core.xml

Author, dates, title — Dublin Core metadata

xl/sharedStrings.xml

Deduplicated text values (compression!)

Content and presentation are stored separately.

This is the same principle behind a semantic layer in data engineering. Modern tools like dbt metrics, Looker's LookML, and Snowflake's semantic layer all formalize this same separation — defining what data means, independent of how it's stored or displayed.

The Semantic Layer Principle

The same three-layer separation appears everywhere:

Layer	In Office Files	In a Data Lake	In a BI Stack
Storage	XML files in ZIP	Parquet files in S3	Database tables
Semantic	Column names, shared strings	Glue Catalog / dbt	LookML / semantic model
Presentation	styles.xml, theme	N/A at storage layer	Dashboard / report

The "it's just a ZIP" pattern extends further than Office:

.odt/.ods (LibreOffice), .epub (ebooks), .jar/.war (Java), .apk (Android apps) — all ZIP containers with structured contents. If you can unzip it, you can build a pipeline that processes it.

Why Data Engineering is Hard

Pipelines don't fail because the tools are bad.

They fail because real-world data is messy, unpredictable, and changes over time.

What Breaks: And Why It Matters for Policy

-  **Schema drift:** Source adds a column or changes a type. Your pipeline expected 12 columns; now there are 13.
-  **Encoding issues:** Latin-1 vs UTF-8. An accented character in a name breaks your CSV parser.
-  **Null surprises:** A field that was always populated starts coming in empty. Downstream aggregation silently wrong.
-  **Duplicate records:** Two agencies report the same person slightly differently. Is "J. Smith" the same as "John Smith"?
-  **Date format chaos:** 01/02/2024 — January 2nd or February 1st? Depends on who sent the file.

For policy: bad data quality in pipelines has led to incorrect benefits calculations, misallocated funding, and flawed program evaluations. Data engineering isn't just plumbing, it's the foundation of analytical trust.

What the Crawler Gets Wrong

Automated schema inference is a starting point, not a final answer. Common failures:

ZIP codes inferred as integers: leading zeros stripped (07302 → 7302)

Phone numbers treated as numbers

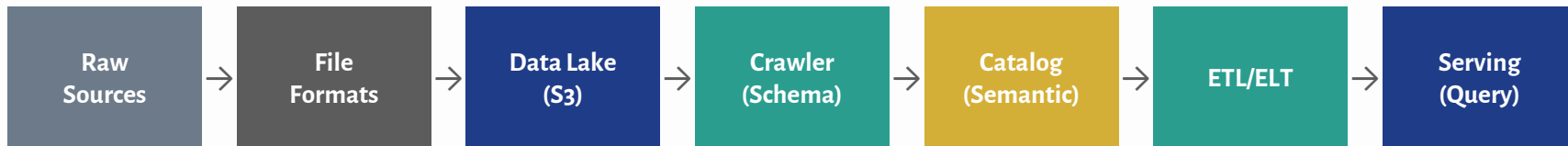
FIPS codes treated as integers instead of strings

"Yes" / "No" fields not recognized as boolean

Dates in ambiguous formats guessed incorrectly

The lesson: Human review of inferred schemas is always necessary. This connects back to the data dictionary — automated or not, someone must verify that the schema matches reality. The `mini_glue_crawler` demo showed exactly where these judgment calls happen.

Putting It All Together



Each step is a choice with tradeoffs. Knowing those tradeoffs — not just the tools — is what makes you effective.

- ✓ Data engineering answers: "How does data get into a queryable state?"
- ✓ File format choices have real consequences for cost, speed, and flexibility
- ✓ Schema-on-read vs schema-on-write is a governance design decision
- ✓ OLTP (rows) vs OLAP (columns) is the fundamental storage tension
- ✓ The semantic layer separates meaning from storage; Census dictionaries to dbt
- ✓ Automated tools help, but human judgment about quality is irreplaceable

Today's Lab: Building a Data Pipeline in AWS

What you'll build:

- 1 Upload CSV files to S3 and crawl them with Glue
- 2 Query the cataloged data with Athena SQL
- 3 Upload JSON and convert between structured and semi-structured formats
- 4 Compare schema-on-read vs schema-on-write approaches

Connects to:

- The mini Glue crawler — you've seen the inference logic
- The file format discussion — you'll see the cost difference
- The format conversion concepts — you'll do it in SQL